

PROCESSAMENTO DE IMAGENS - 2D

FAESA

ENGENHARIA DA COMPUTAÇÃO

PROF. DANIEL LUIS COSMO

Definição

- Computação gráfica envolve a ciência e a arte de comunicação visual utilizando displays de computadores e dispositivos de interação com o usuário.

Exemplos:

- Telas de aplicativos
- Animações
- Video games
- Representações realistas de procesos físicos

- Computação gráfica é um tópico multidisciplinar, envolvendo as áreas de:
 - Física
 - Matemática
 - Percepção humana
 - Interação homem-máquina
 - Engenharia
 - Computação
 - Design gráfico
 - Arte

- Computação gráfica x Visão computacional

- Computação gráfica: Informação flui do computador para os usuários externos
- Visão computacional: Informação flui do ambiente externo para os computadores:

Computador recebe e reconhece imagem da planta, as bordas entre folhas, flores e fundo

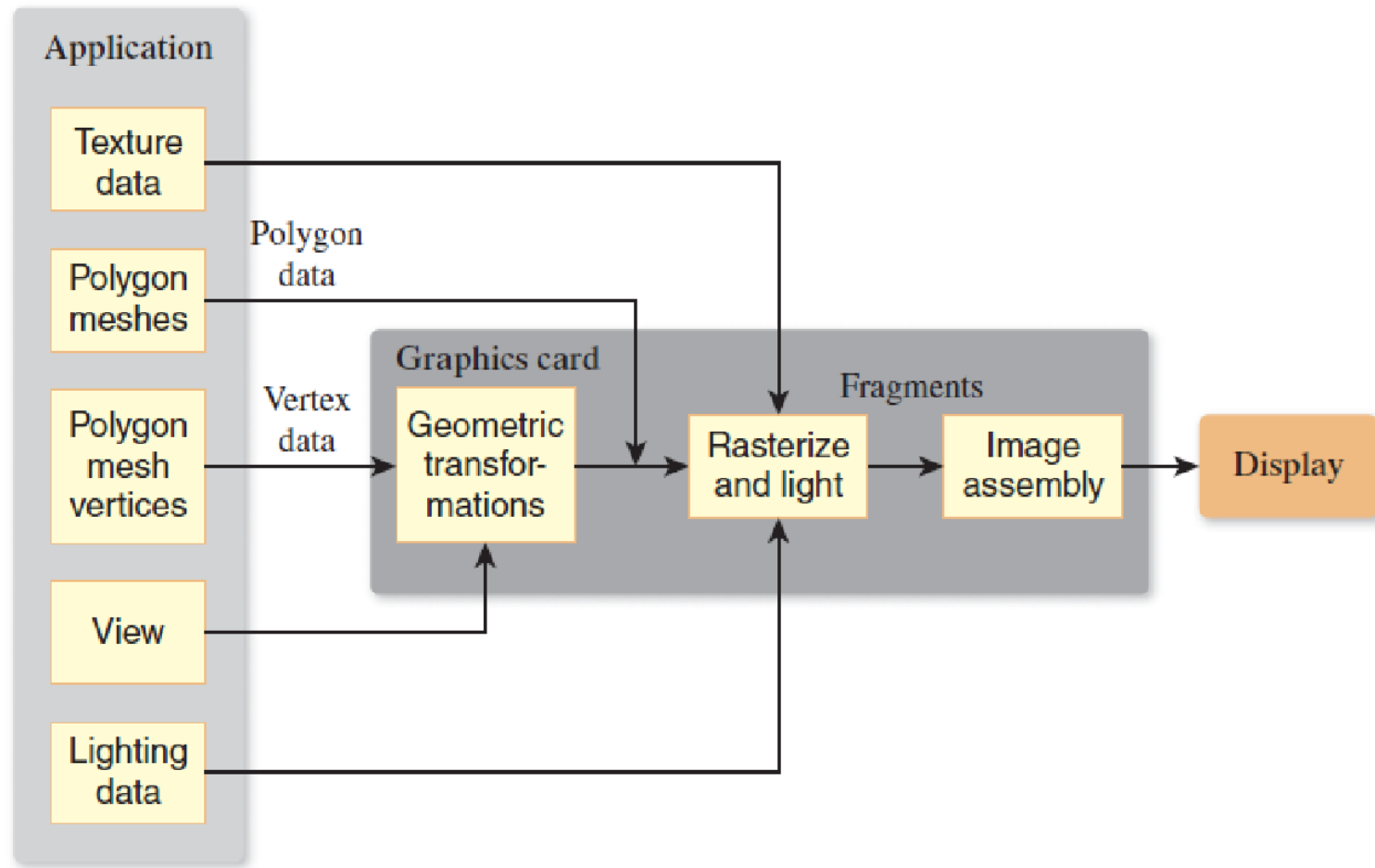


Computador renderiza uma imagem cartunizada da planta em seu display

- Computação gráfica x Visão computacional
 - Cheat-maker brags of computer-vision auto-aim that works on “any game” ([link](#))



Pipeline Gráfico



- **Procesamento da geometria dos vertices:**
 - Responsável por tomar uma descrição geométrica de um objeto, normalmente expressa em termos das localizações de certos vértices de uma malha poligonal, juntamente com certas transformações a serem aplicadas a esses vértices, e computar as posições reais dos vértices depois que eles foram transformados. Os polígonos da malha, que são definidos em termos dos vértices também são transformados.
- **Processamento de triângulos (rasterização):**
 - Pega os triângulos da malha e uma especificação para uma câmera virtual cuja visão estamos renderizando, e processa os polígonos um por um em um processo chamado rasterização, para converter eles de uma representação de geometria contínua (3D) para a geometria da tela pixelizada (2D).

- Textura e iluminação
 - Aos fragmentos resultantes (pixels ou porções de pixels que pertencem ao triângulo e podem eventualmente aparecer na tela se não estiverem obscurecidos por algum outro fragmento) são atribuídas cores com base na iluminação da cena e as texturas que foram atribuídas à malha.
- Formação da imagem
 - Se vários fragmentos estão associados à mesma localização de pixel, o primeiro fragmento (o mais próximo do visualizador) é geralmente escolhido para ser desenhado, embora outras operações podem ser realizadas por pixel (por exemplo, cálculos de transparência).

Interação em sistemas gráficos

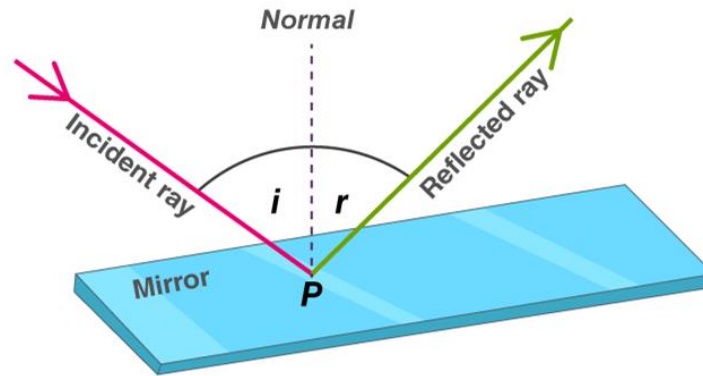
- Interfaces gráficas
 - Duas threads paralelas
 1. Controla o processamento de dados do programa principal
 2. Controla a interface gráfica do usuário (GUI).
 - Cada componente da GUI que gera interação com o usuário é associada a uma função de callback no programa principal.

Renderização realista

- Conhecimentos necessários:
 - Uma compreensão da física da luz.
 - Um modelo para os materiais com os quais a luz interage, e para o processo de interação.
 - Um modelo para a forma como capturamos a luz para criar uma imagem.
 - Uma compreensão de como a tecnologia de exibição moderna produz luz.
 - Uma compreensão do sistema visual humano e como ele percebe a entrada luz.
 - E uma compreensão de uma quantidade substancial de matemática usada na descrição de muitas dessas coisas.

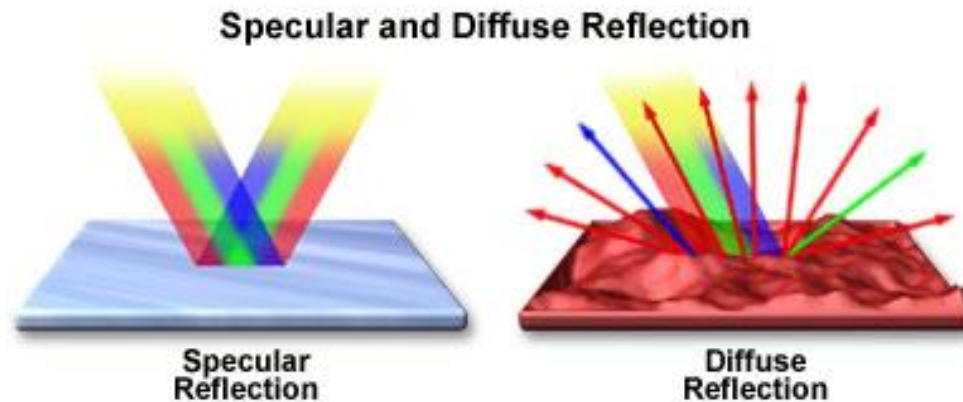
Luz

- A luz se propaga ao longo de raios em linha reta no espaço vazio, parando quando encontra uma superfície.
- A luz reflete em qualquer superfície brilhante que encontra, seguindo um modelo de "ângulo de incidência igual ao ângulo de reflexão", ou é absorvido pela superfície, ou alguma combinação dos dois (por exemplo, 40% absorvido, 60% refletido).

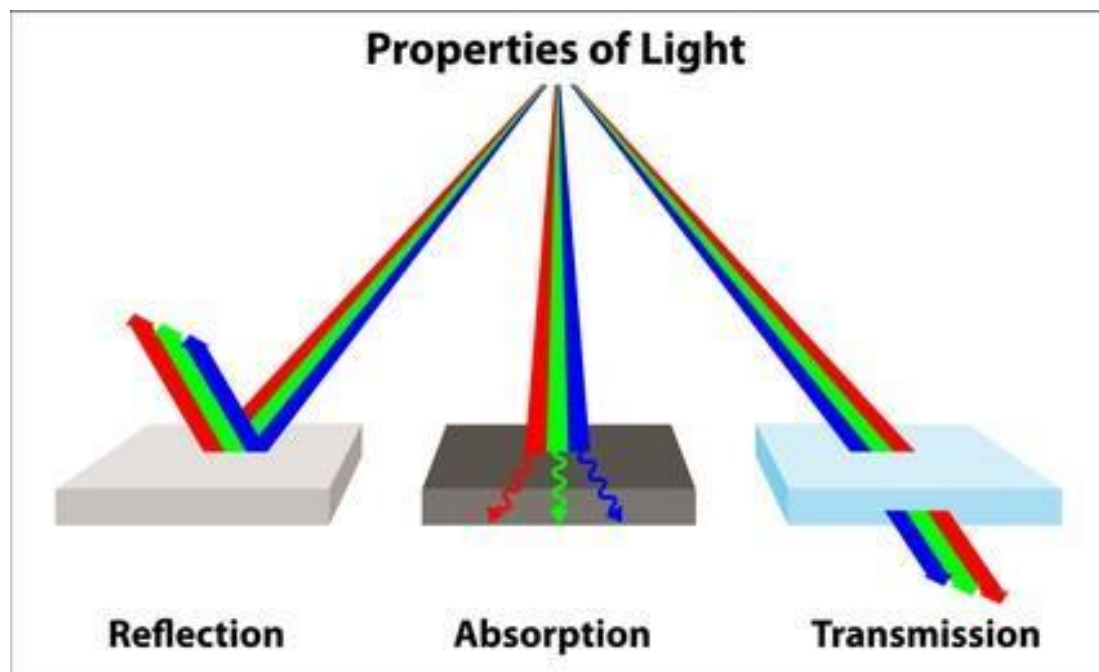


Materias

- A luz só interage na superfície dos materiais.
- A superfície de um material pode ser especular, difusa, ou uma combinação das duas.



- Dependendo do material, ele pode refletir, absorver ou transmitir a luz.



Captura da luz

- O sensor de uma câmera e o olho humano respondem à luz de maneira semelhante: Eles acumulam energia luminosa por algum período de tempo e, em seguida, relatam a energia acumulada.
- Simulando tal sensor (ou célula), portanto, envolve a computação de uma integral da luz que entra sobre a área do sensor.

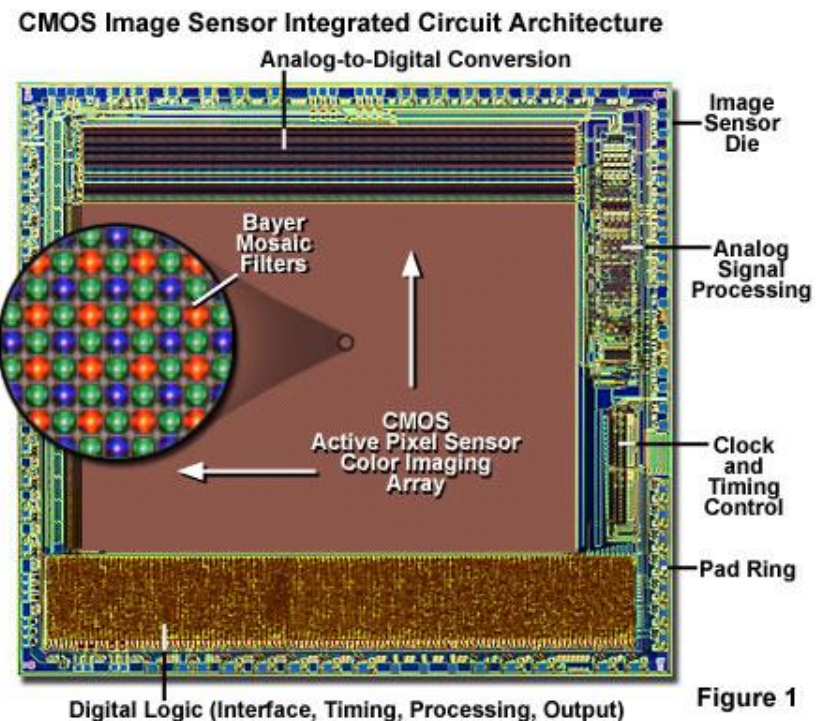
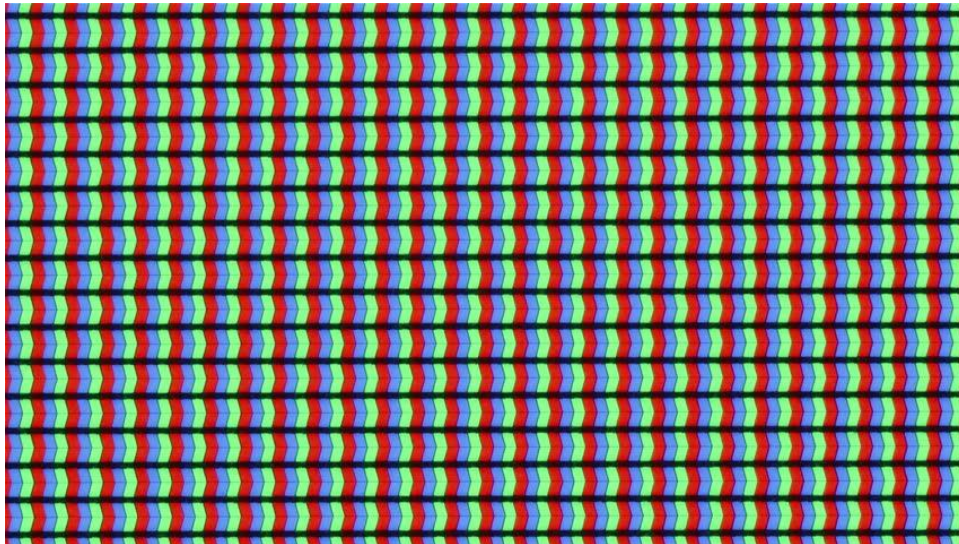


Figure 1

Display

- Monitores modernos são divididos em unidades chamadas de pixels.
- Cada pixel pode emitir uma combinação de três cores diferentes ,vermelho, verde e azul (RGB), normalmente possuindo 256 níveis de intensidade diferentes (8 bits ou 2 hexadecimais).
- 0xFF00CC significa R = FF ou 256 decima, G = 00 ou 0 decima e B = CC ou 204 decimal



Matemática

- A matemática usada na computação gráfica envolve:
 - Trigonometria
 - Álgebra Linear
 - Integrais e derivadas
 - Noções de topologia geométricas, como superfícies em três dimensões e curvaturas.

Transformações lineares

- Transformações 2D lineares tem a forma:

$$T : \mathbf{R}^2 \rightarrow \mathbf{R}^2, \quad T(\mathbf{v} + \alpha\mathbf{w}) = T(\mathbf{v}) + \alpha T(\mathbf{w})$$

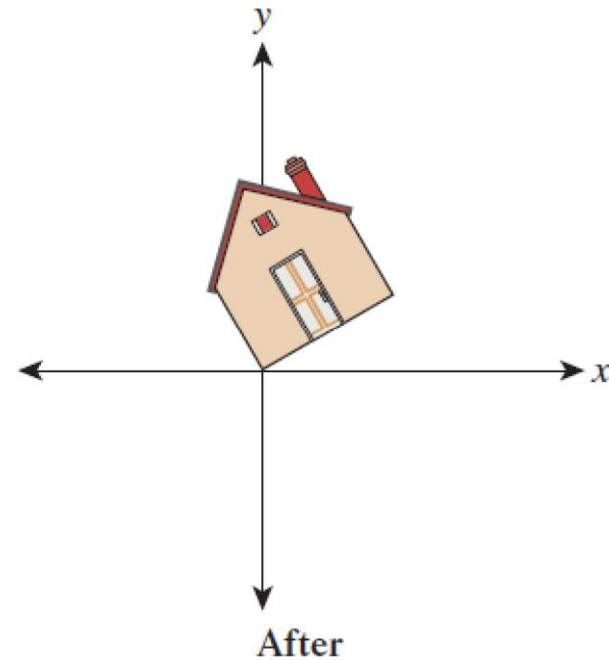
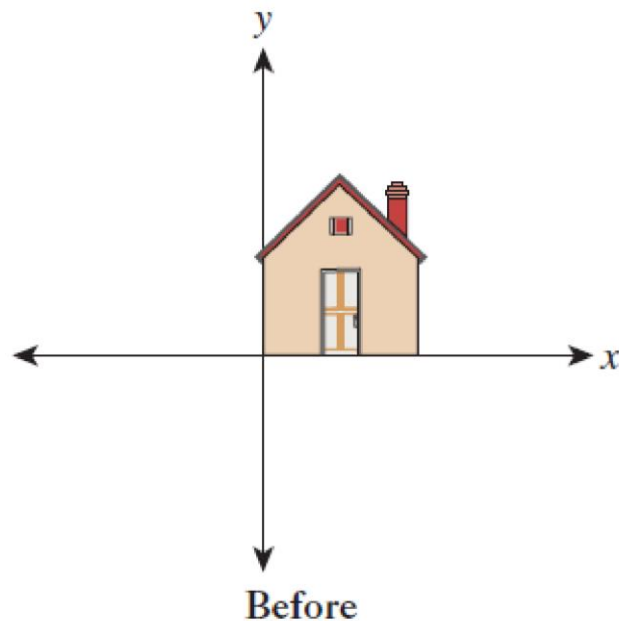
- Tais transformações preservam as linha e mantém a origem no mesmo lugar.
- Transformações de rotação e escala são exemplos de transformações 2D lineares, porém translações não são lineares.
- Uma transformação linear 2D pode ser representada por uma multiplicação matricial

$$\mathbf{v} \mapsto \mathbf{M}\mathbf{v}$$

onde $\mathbf{M}$ é uma matriz 2x2.

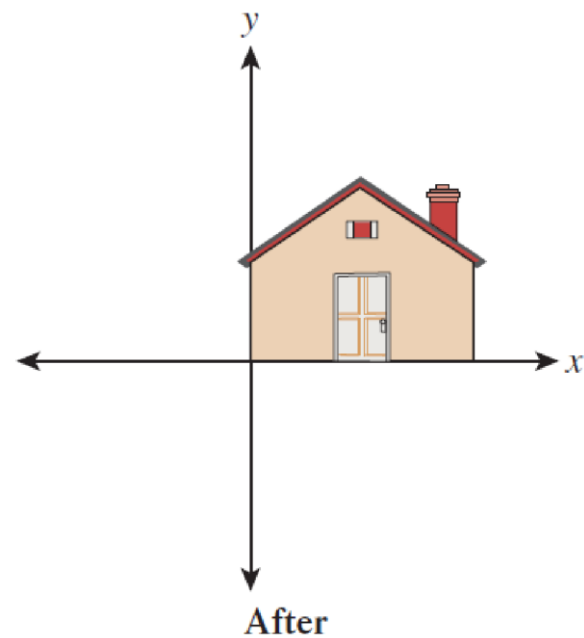
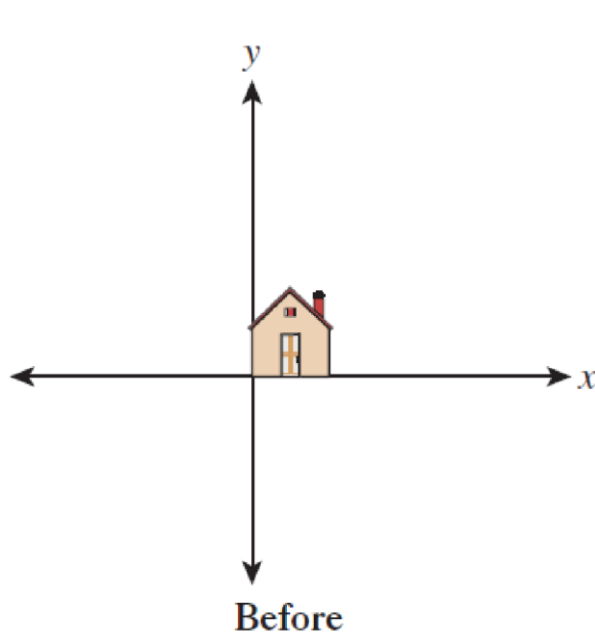
- Rotação:

$$T_1 : \mathbf{R}^2 \rightarrow \mathbf{R}^2 : \begin{bmatrix} x \\ y \end{bmatrix} \mapsto \mathbf{M}_1 \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \cos 30^\circ & -\sin 30^\circ \\ \sin 30^\circ & \cos 30^\circ \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



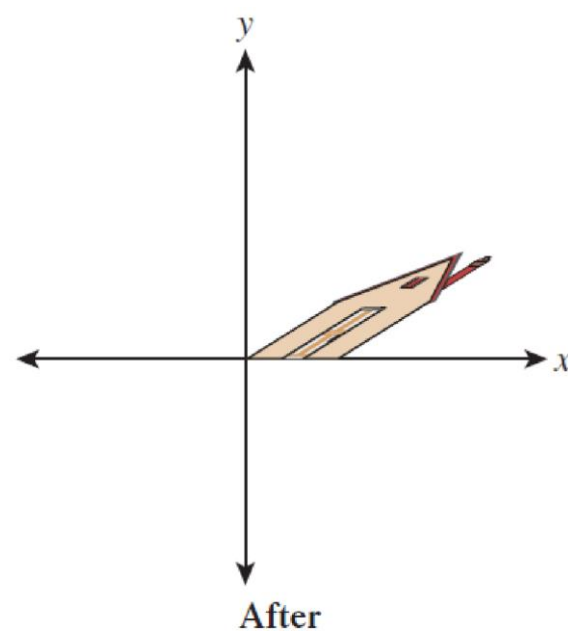
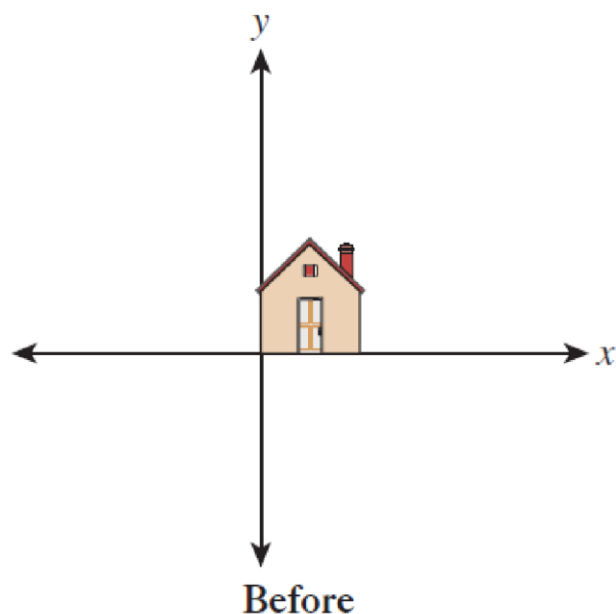
- Escala:

$$T_2 : \mathbf{R}^2 \rightarrow \mathbf{R}^2 : \begin{bmatrix} x \\ y \end{bmatrix} \mapsto \mathbf{M}_2 \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 3x \\ 2y \end{bmatrix}$$



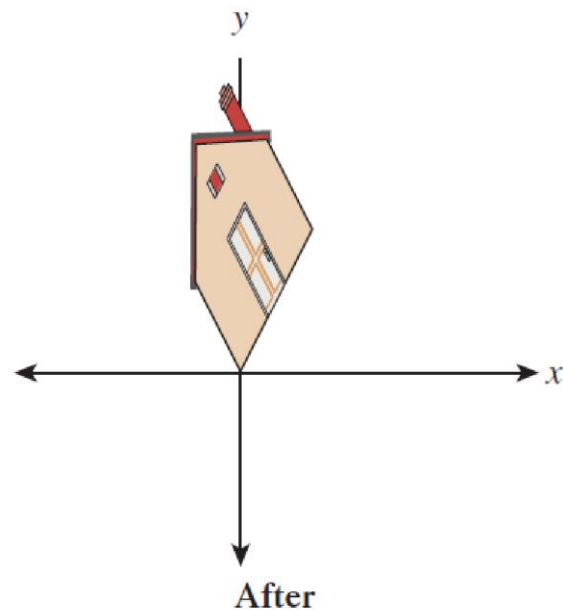
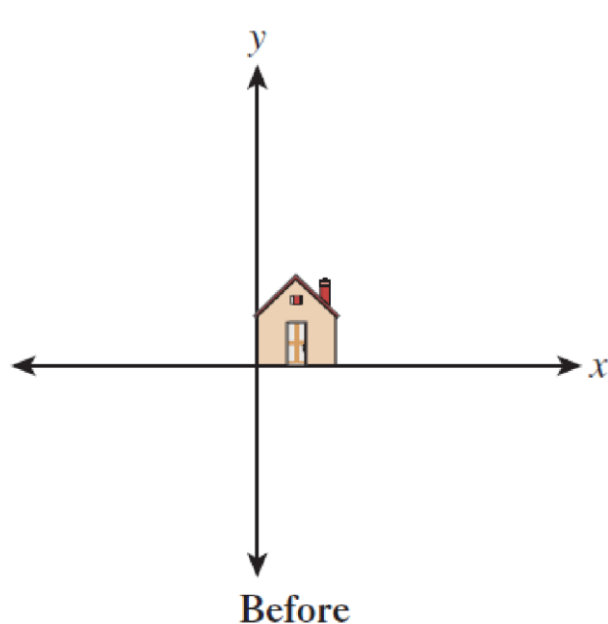
- Cisalhamento:

$$T_3 : \mathbf{R}^2 \rightarrow \mathbf{R}^2 : \begin{bmatrix} x \\ y \end{bmatrix} \mapsto \mathbf{M}_3 \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + 2y \\ y \end{bmatrix}.$$



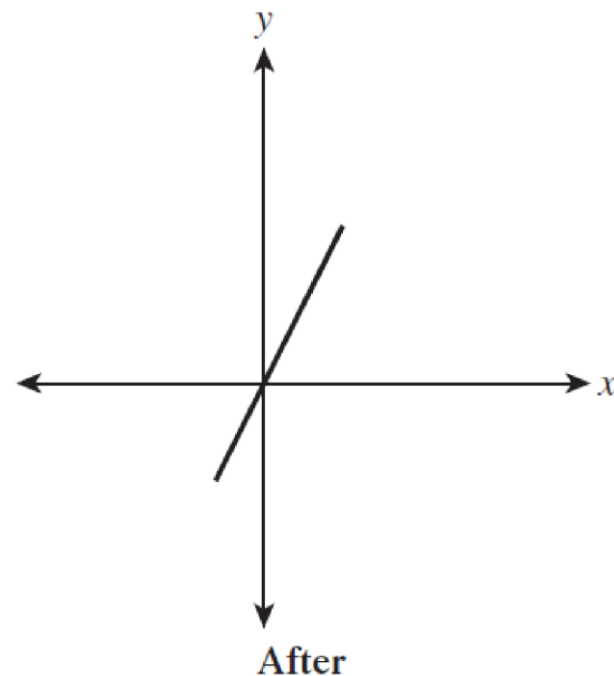
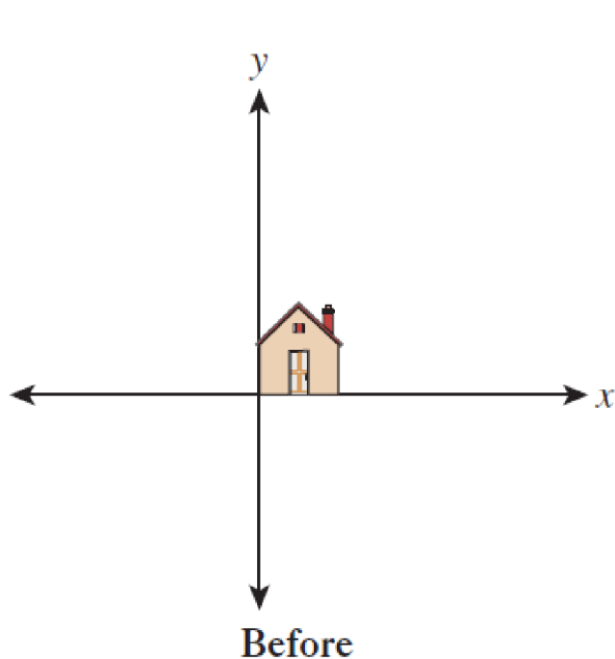
- Transformação geral:

$$T_4 : \mathbf{R}^2 \rightarrow \mathbf{R}^2 : \begin{bmatrix} x \\ y \end{bmatrix} \mapsto \mathbf{M}_4 \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

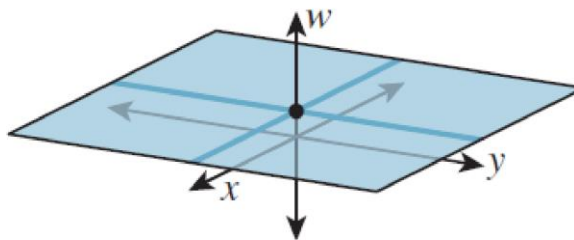


- Transformação degenerada:

$$T_5 : \mathbf{R}^2 \rightarrow \mathbf{R}^2 : \begin{bmatrix} x \\ y \end{bmatrix} \mapsto \begin{bmatrix} 1 & -1 \\ 2 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x - y \\ 2x - 2y \end{bmatrix}$$



- Translação:
- Para que a translação possa ser realizada como uma transformação linear, devemos usar um plano ($w=1$) no espaço xyw 3D.



- Neste plano, os pontos são representados com 3 coordenadas, sendo que a terceira deve ser sempre igual a 1.

$$\begin{bmatrix} a & b & c \\ d & e & f \\ p & q & r \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

- Para que um vetor seja transformado e mantenha a terceira coordenada igual a 1, é necessário que $p = q = 0$ e $r = 1$.

$$\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

- A matriz 2×2 $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ é responsável por aplicar as transformações lineares 2D vistas anteriormente, enquanto o vetor 2×1 $\begin{bmatrix} c \\ f \end{bmatrix}$ é responsável pela translação.

$$\begin{bmatrix} 1 & 0 & c \\ 0 & 1 & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + c \\ y + f \\ 1 \end{bmatrix}$$

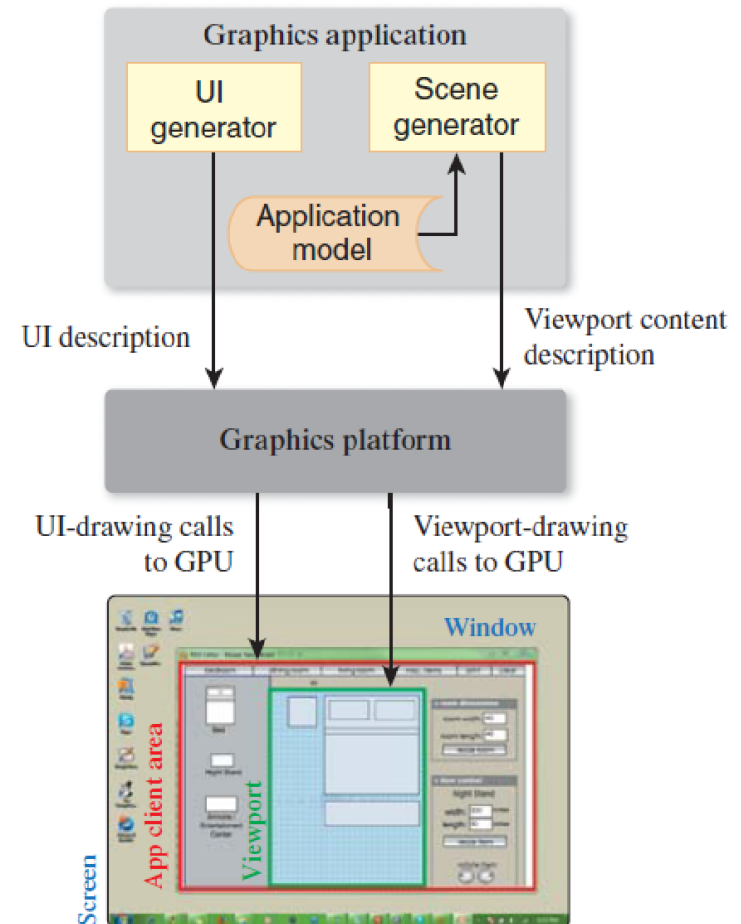
- As transformações que juntam transformações lineares e translações são chamadas de transformações afins.

Introdução WPF

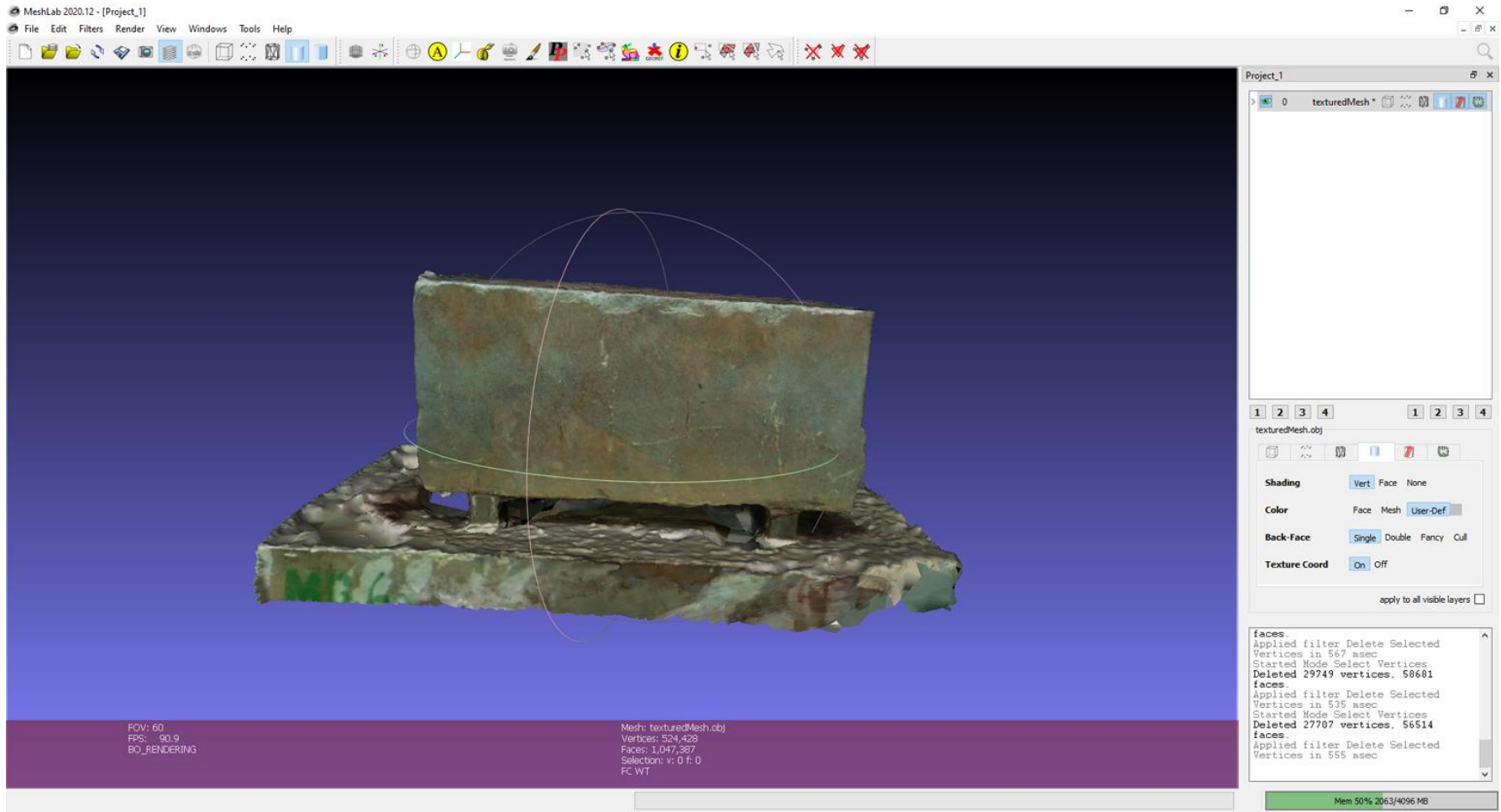
- O WPF (Windows Presentation Foundation) foi escolhido porque é uma das poucas plataformas gráficas modernas que suporta aplicativos 2D e 3D, fornecendo interface de usuário, bem como funcionalidade de renderização usando um modelo de programação consistente.
- O WPF utiliza a linguagem declarativa XAML para desenho do gráficos e a linguagem procedural C# como code-behind.

Visão geral do WPF 2D

- Aplicação gráfica: É responsável por gerar a área gráfica do aplicativo. Controla tanto a interface do usuário como a renderização da cena. Um modelo de aplicação (AM) é responsável por atualizar a cena de acordo com a interação do usuário.
- Plataforma gráfica: Normalmente é uma API que transmite as informações da aplicação gráfica para a GPU



- Exemplo: MeshLab



Camadas do WPF

- API: A camada central é um conjunto de classes que fornecem todas as funcionalidades do WPF (C# ou visual basic). Pode ser usada para especificar aparência e comportamento do aplicativo
- XAML: A camada intermediária fornece uma maneira alternativa de especificar um grande subconjunto de funcionalidade da API, por meio da linguagem declarativa XAML.
- A camada mais alta da interface do aplicativo WPF compreende os utilitários que designers e engenheiros podem usar para gerar XAML.

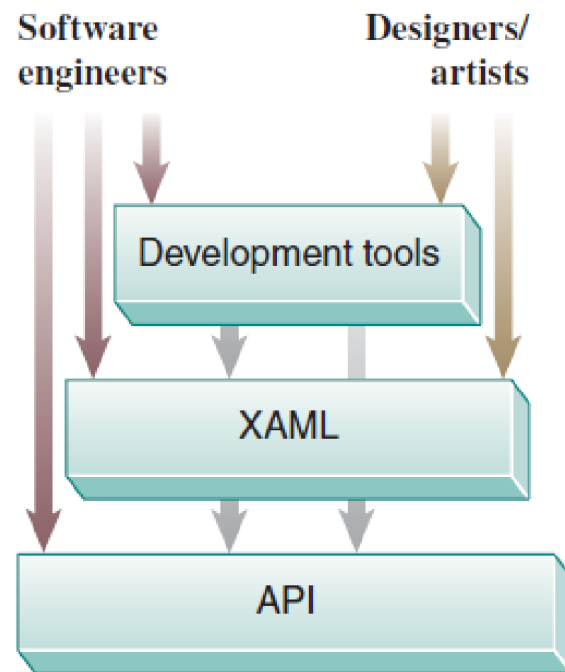
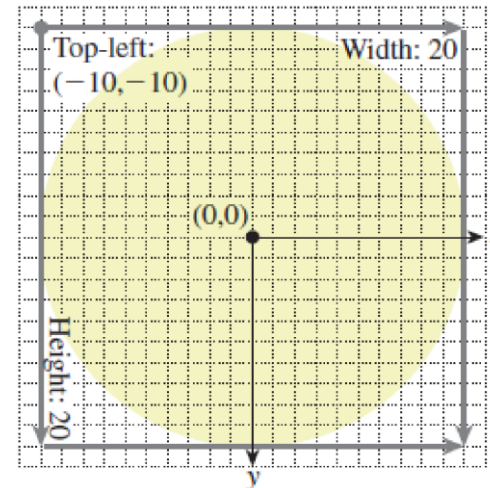
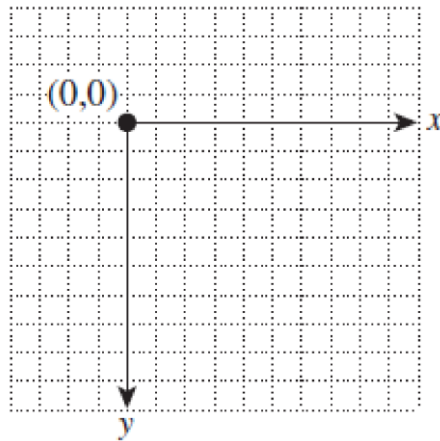


Figure 2.5: WPF application/developer interface layers.

Exemplo 2D WPF - Relógio animado

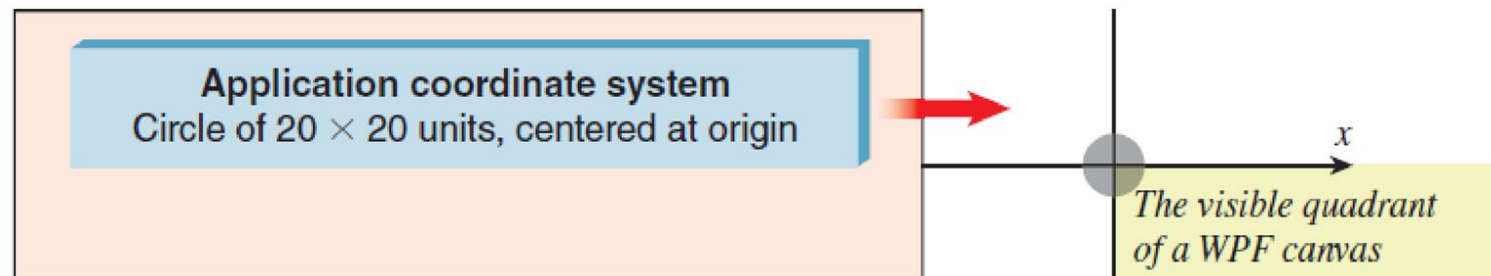
- Na sequencia dos slides, iremos criar um relógio 2D no WPF, e com isso aprender sobre transformações básicas, criação de templates e animações usando XAML.
- Iremos usar a classe “Canvas” para desenhar os elementos gráficos. Esta classe permite posicionar explicitamente elementos filho com o uso de coordenadas que são relativas à área da classe pai.

- Iremos primeiro criar um círculo, usando a classe “Ellipse”, baseado em um sistema de coordenadas abstrato.

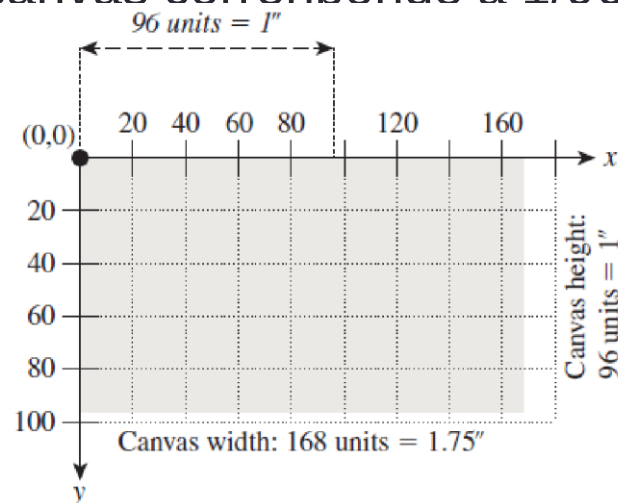


```
<Canvas ... >  
  <Ellipse  
    Canvas.Left="-10.0" Canvas.Top="-10.0"  
    Width="20.0" Height="20.0"  
    Fill="lightgray" />  
</Canvas>
```

- Observem que o círculo fica pequeno e fora de posição na aplicação final.



- A coordenada (0,0) se refere ao canto superior esquerdo do Canvas.
- Cada unidade do canvas corresponde a 1/96 polegadas no display.



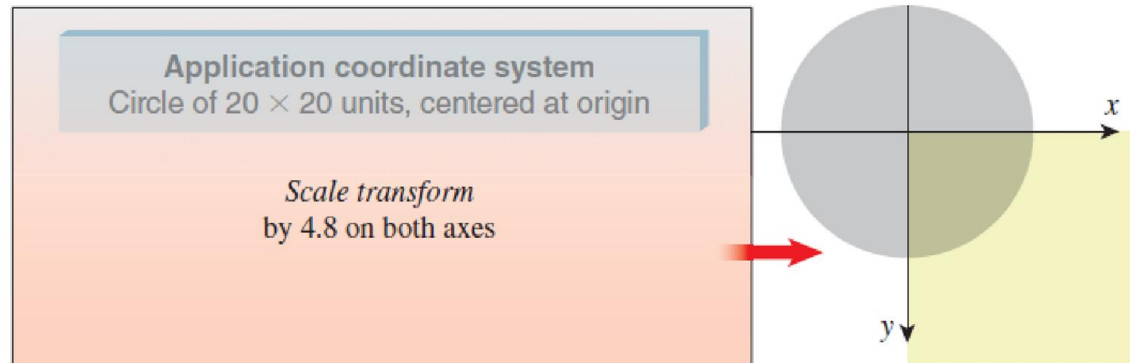
- O círculo deve possuir uma polegada e ser mostrado inteiramente no canto superior esquerdo do aplicativo.
- Para obter tal resultado, iremos aplicar duas transformações no Canvas: uma mudança de escala e uma translação.
- Primeiro a mudança de escala.

```
<Canvas ... >

  <!-- THE SCENE -->
  <Ellipse ... />

  <!-- DISPLAY TRANSFORMATION -->
  <Canvas.RenderTransform>
    <!-- The content of a RenderTransform is a TransformGroup
         acting as a container for ordered transform elements. -->
    <TransformGroup>
      <!-- Use floating-point scale factors:
           1.0 to represent 100%, 0.5 to represent 50%, etc. -->
      <ScaleTransform ScaleX="4.8" ScaleY="4.8"
                     CenterX="0" CenterY="0"/>
    </TransformGroup>
  </Canvas.RenderTransform>

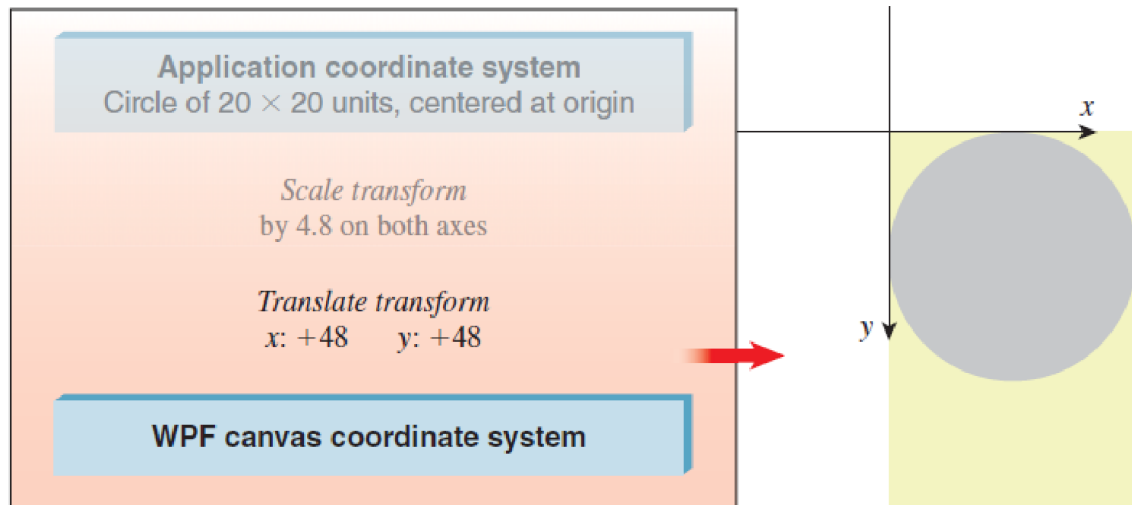
</Canvas>
```

- Em seguida a translação.

```
<Canvas ... >
  <!-- THE SCENE -->
  <Ellipse ... />

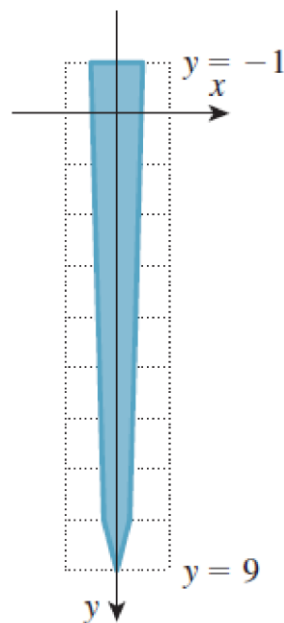
  <!-- THE DISPLAY TRANSFORM -->
  <Canvas.RenderTransform>
    <TransformGroup>
      <ScaleTransform ScaleX="4.8" ScaleY="4.8" ... />
      <TranslateTransform X="48" Y="48" />
    </TransformGroup>
  </Canvas.RenderTransform>
</Canvas>
```



- Com isso, o primeiro elemento do relógio está pronto.

- Iremos agora criar um template para os ponteiros do relógio.
- Os ponteiros serão feitos utilizando a classe “Polygon”.

```
<Polygon  
  Points="-0.3, -1   -0.2, 8   0, 9   0.2, 8   0.3, -1"  
  Fill="Navy" />
```



- Os templates são definidos na seção de recursos (“resources”) do Canvas.

```
<Canvas ... >

  <!-- First, we define reusable resources,
    giving each a unique key: -->
  <Canvas.Resources>
    <ControlTemplate x:Key="ClockHandTemplate">
      <Polygon ... />
    </ControlTemplate>
  </Canvas.Resources>

  <!-- THE SCENE -->
  <Ellipse ... />

  <!-- THE DISPLAY TRANSFORM -->
  <Canvas.RenderTransform> ... </Canvas.RenderTransform>
</Canvas>
```

- Os elementos gráficos do WPF são chamados de controles (“control”).

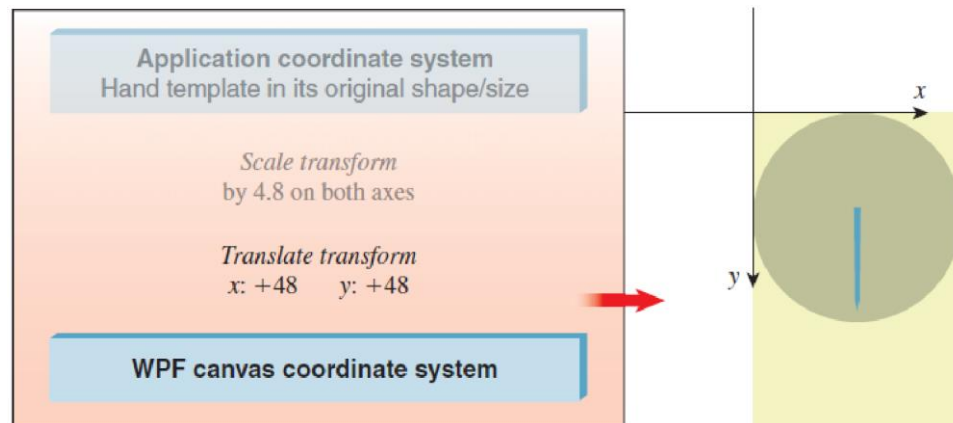
- Utilizaremos o template definido anteriormente para instanciar o ponteiro dos minutos.

```
<!-- THE SCENE -->

<!-- The clock face -->
<Ellipse ... />

<!-- The minute hand: -->
<Control Name="MinuteHand"
  Template="{StaticResource ClockHandTemplate}"/>
```

- O ponteiro é automaticamente transformado pelas transformações definidas anteriormente.



- O ponteiro das horas será instanciado usando o mesmo template.
- Iremos aplicar uma transformação de escala para que o ponteiro das horas fique com uma aparência diferente do ponteiro dos minutos.
- E mais uma transformação de rotação para que ele aponte para outra direção.

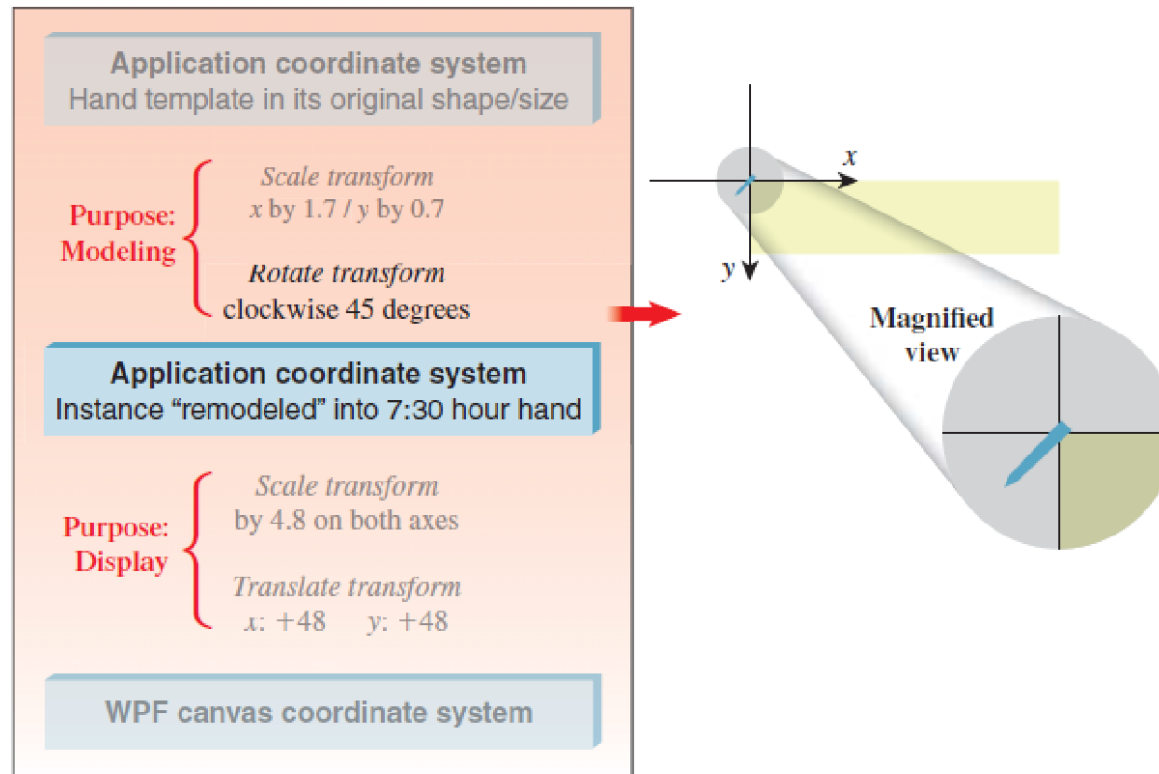
```
<!-- The hour hand: -->  
<Control Name="HourHand" Template="{StaticResource ClockHandTemplate}">  
  <Control.RenderTransform>  
    <TransformGroup>  
      <ScaleTransform ScaleX="1.7" ScaleY="0.7" CenterX="0" CenterY="0"/>  
      <RotateTransform Angle="45" CenterX="0" CenterY="0"/>  
    </TransformGroup>  
  </Control.RenderTransform>  
</Control>
```

- Observem que o código possui duas seções de transformação, uma que age no canvas inteiro e outra que age somente no ponteiro de horas.

```
<Canvas ... >
  <!-- RESOURCES ATTACHED TO THE CANVAS -->
  <Canvas.Resources>
    <ControlTemplate x:Key="ClockHandTemplate">
      <Polygon ... />
    </ControlTemplate>
  </Canvas.Resources>

  <!-- THE SCENE -->
  <!-- The clock face: -->
  <Ellipse ... />
  <!-- The minute hand: -->
  <Control Name="MinuteHand"
    Template="{StaticResource ClockHandTemplate}"/>
  <!-- The hour hand: -->
  <Control Name="HourHand"
    Template="{StaticResource ClockHandTemplate}">
    <Control.RenderTransform>
      The modeling transform for the hour hand should be here.
    </Control.RenderTransform>
  </Control>
  <!-- THE DISPLAY TRANSFORM -->
  <Canvas.RenderTransform>
    The display transform for the scene should be here.
  </Canvas.RenderTransform>
</Canvas>
```

- Resultado final do ponteiro de horas.



- O próximo passo é animar os ponteiros. O WPF consegue gerar animações simples direto no XAML.
- Primeiro, vamos apontar o ponteiro de horas para a posição das 12:00.

```
<!-- Rotate into 12 o'clock default position -->  
<RotateTransform Angle="180"/>
```

- Depois, vamos criar outra transformação de rotação que será usada na animação.

```
<TransformGroup>  
  <ScaleTransform ScaleX="1.7" ScaleY="0.6" />  
  <!-- Rotate into 12 o'clock default position -->  
  <RotateTransform Angle="180"/>  
  <!-- Additional rotation for animation to show actual time: -->  
  <RotateTransform x:Name="ActualTimeHour" Angle="0"/>  
</TransformGroup>
```

- O WPF fornece um tipo de elemento de animação para cada tipo de dados que podemos querer automatizar. Para controlar o ângulo de rotação, que é um tipo “double”, usamos o tipo de elemento “DoubleAnimation”.

```
<DoubleAnimation  
    Storyboard.TargetName="ActualTimeHour"  
    Storyboard.TargetProperty="Angle"  
    From="0.0" To="360.0" Duration="1:00:00.0"  
    RepeatBehavior="Forever"  
>
```

- Antes da animação funcionar, devemos escolher um trigger para ela começar. Para isso, usamos a propriedade “triggers” do canvas.
- As animações ficam dentro de um elemento chamado “Storyboard” do WPF.

```
<Canvas ... >
```

The specification of the clock scene should be located here.

```
<Canvas.Triggers>
```

```
<EventTrigger RoutedEvent="FrameworkElement.Loaded">
```

```
<BeginStoryboard>
```

```
<Storyboard>
```

```
<DoubleAnimation
```

```
Storyboard.TargetName="ActualTimeHour"
```

```
Storyboard.TargetProperty="Angle"
```

```
From="0.0" To="360.0"
```

```
Duration="01:00:00.00" RepeatBehavior="Forever" />
```

*Two more DoubleAnimation elements should be located
here to animate the other clock hands.*

```
</Storyboard>
```

```
</BeginStoryboard>
```

```
</EventTrigger>
```

```
</Canvas.Triggers>
```

```
</Canvas>
```

- Do mesmo modo que animamos o ponteiro das horas, podemos animar o ponteiro dos minutos, finalizando o exemplo do relógio.